جامعة دجلة

Computer organization and Architecture

قسم تقنيات الحاسوب

المرحلة الثانية

الفصل الأول

مدرس المادة:م م محمد اسماعيل صحن

Introduction to computer system

What is a Computer System?

- An electronic system that:

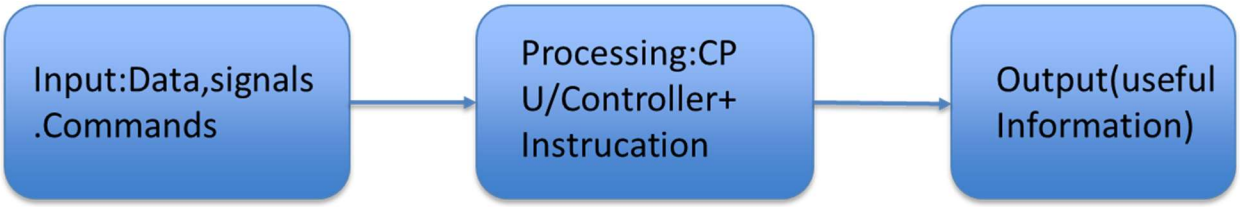
1. Accepts input

2. Processes data using instructions

3. Produces output (useful information)

Components:

- **Hardware (physical devices)**
- **•Software (programs and instructions)**



•Explanation:

- Input: Keyboard, sensors, mouse, or other data sources.

- Processing: CPU, microcontroller, or control unit applying instructions.

- Output: Screen, actuator, printer, or other useful information output devices.

why using computer:

We use computers because they provide unmatched speed, accuracy, and automation for processing information.

In short, the reasons are:

Speed & Efficiency: They execute billions of instructions per second, automating complex, repetitive tasks much faster than humans (e.g., simulations, manufacturing).

Accuracy: They perform mathematical and logical calculations with guaranteed numerical precision, crucial for finance, engineering, and science.

Data Management: They reliably store, process, and instantly retrieve massive volumes of data, powering global databases and the internet.

Communication: They facilitate instant global communication (email, video) and networking.

Creativity & Learning: They provide indispensable tools for design, entertainment, and education.

computer generation over time:

First Generation

Approximate Era: 1940–1956 Defining Technology: Vacuum Tubes Primary Purpose of Use: Military and Scientific Calculations: Used for complex mathematical tasks, such as calculating artillery trajectories (ENIAC), code-breaking (Colossus), and early tomic research. Served as large-scale basic data processing machines.



Second Generation

Approximate Era: 1956–1963 Defining Technology: Transistors Primary Purpose of Use: Beginning of Commercial Computing: Enabled computers to be used for commercial and research purposes, leading to the true start of adoption in industries. Used for large-scale business data processing and scientific simulations.



Third Generation

Approximate Era: 1964–1971 Defining Technology: Integrated Circuits (ICs) Primary Purpose of Use: Standardization and Access: Introduced operating systems and allowed for efficient multitasking and resource management. Standardized business computing (e.g., IBM System/360) and democratized access in smaller businesses, universities, and research labs.



Fourth Generation

Approximate Era: 1971–Present Defining Technology: Microprocessors Primary Purpose of Use: Personal Computing and Networking: Created the Personal Computer (PC) revolution, making computers affordable and widely accessible for households and small businesses. Focused on general-purpose tasks like word processing, spreadsheet analysis, and gaming. Enabled the development of computer networks and the Internet.



Fifth Generation:

Approximate Era: Present & Beyond **Defining Technology:** Artificial Intelligence (AI) and Quantum Computing **Primary Purpose of Use: Intelligent Systems and Complex Problem-Solving**: Aimed at developing systems capable of **learning, self-organization, and natural language processing**. Used for advanced applications like **AI diagnostics** in healthcare, **algorithmic trading**, **self-driving vehicles**, and solving massive **scientific modeling** challenges.



computer generation classified by component:

Computer classification based on the number of users:

primarily relates to the size and processing power of the system, which determines its

capacity to handle simultaneous tasks and user sessions.

This classification divides computers into two main groups: Single-User and Multi-User systems.

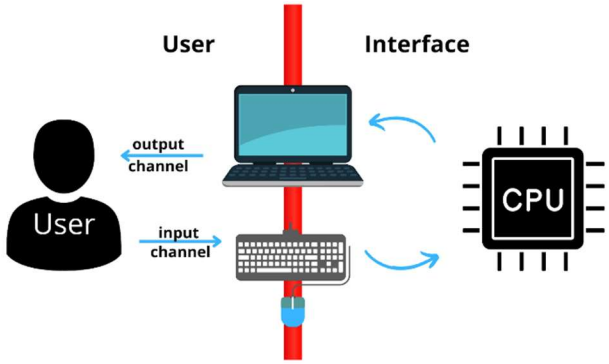
Classification by Number of Users

Single-User Systems

These machines are designed to meet the processing and memory needs of a single individual at a time.

Microcomputers: This is the most common type and includes Desktop PCs, Laptops, Smartphones, and Tablets. They are general-purpose machines intended for everyday tasks like browsing, word processing, and light computing.

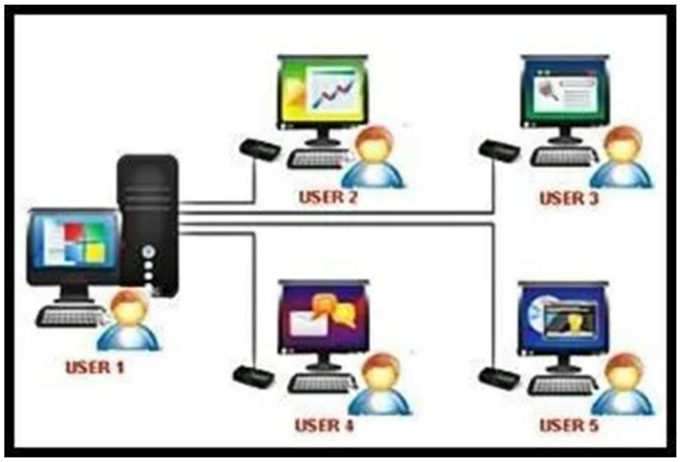
Workstations: A high-performance computer designed for a single user, intended for professional or technical work (e.g., 3D graphics rendering, video editing, CAD/CAM, and complex scientific simulations).



Multi-User Systems

These powerful computers are designed to serve dozens, hundreds, or even thousands of users concurrently, often handling massive amounts of data and complex application workloads.

Minicomputers (Midrange Servers):

Smaller 

than mainframes but still powerful, designed to support multiple users (typically tens to hundreds) within a department or a mid-sized organization. They are often used as web servers, file servers, or database servers.

Mainframe Computers:

Extremely large and powerful computers designed for high-volume, transactional processing (e.g., banking, airline reservations, and census data). They can support hundreds or thousands of users simultaneously and are known for their high reliability and security.

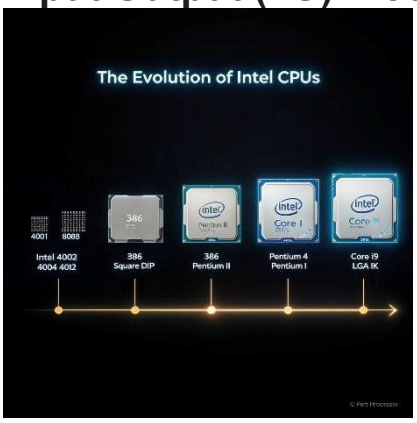


Supercomputers:

The fastest and most powerful machines, designed to perform extremely complex calculations and process data at massive speeds. While they may not have thousands of interactive users in the traditional sense, they often run programs for thousands of individual processors simultaneously to solve a single, massive problem (e.g., weather forecasting, nuclear simulation, and scientific modeling).



The major components of a computer system are:

- . The Central Processing Unit (CPU)
- Main Memory.
- Input/Output (I/O) modules,
- 

1. Von Neumann Architecture (Princeton Architecture)

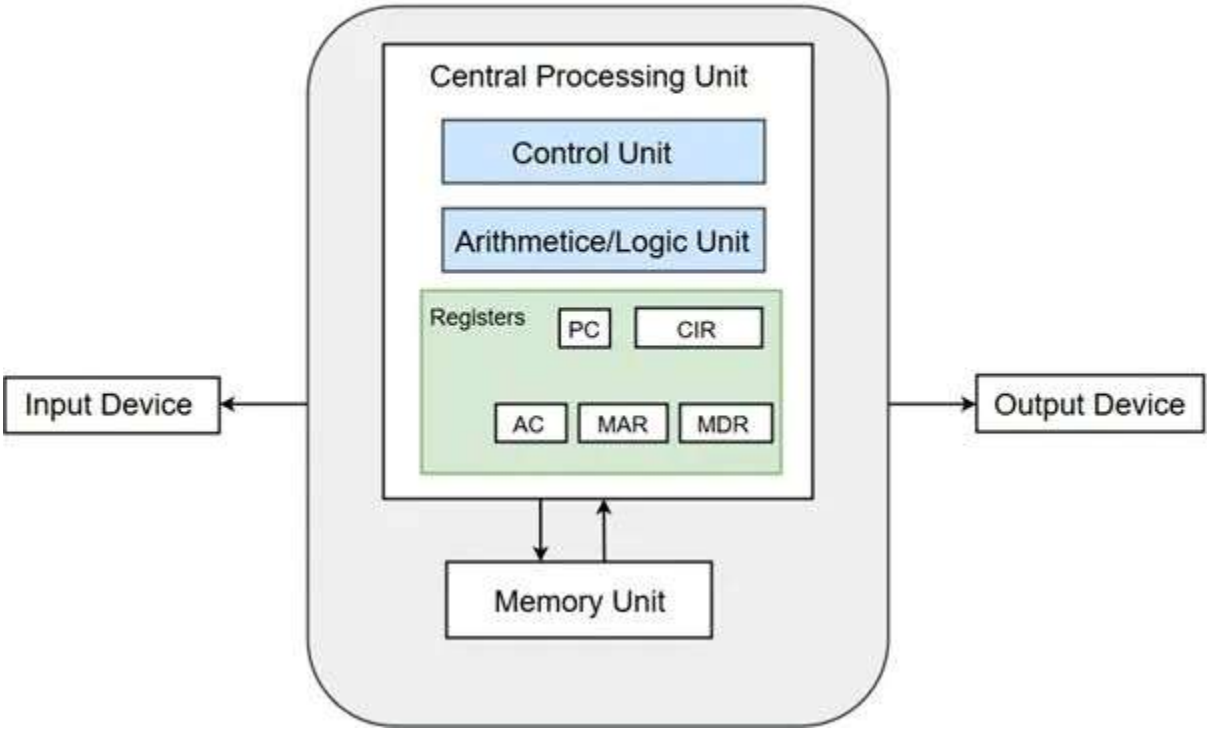
The Von Neumann architecture is the foundational design for almost all modern general-purpose computers, including your PC, laptop, and smartphone. Its key feature is the Stored-Program Concept, where the program's instructions and the data it operates on are stored together in a single, unified memory unit.

Structure and Key Components:

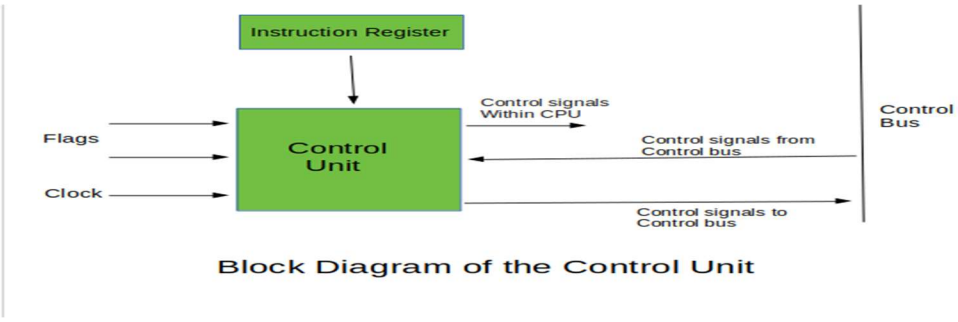
The architecture consists of four main components connected by a single bus system:

Central Processing Unit (CPU):

Von-Neumann Basic Structure:



Control Unit (CU): Directs the entire operation of the processor, fetching and decoding instructions.



Block Diagram of the Control Unit

The CU's role is to ensure that all data movements, arithmetic operations, and instruction execution steps happen at the correct time and in the proper sequence. It does not process or store data itself.

Core Functions:

Fetching and Decoding: It fetches the next instruction from Main Memory and decodes it to determine what operation needs to be performed.

Sequencing: It determines the correct sequence of micro-operations (basic steps) required to execute the instruction (the Fetch-Decode-Execute cycle).

Controlling: It generates the necessary control signals (electrical pulses) and sends them to the other components (ALU, registers, memory, and I/O devices) to make them perform their designated tasks. For example, it sends an *Opcode* to the ALU to specify "Addition" or a *READ* signal to the Memory Unit to fetch data.

Data Flow: It orchestrates the flow of data between the CPU's internal registers, the ALU, and the external bus system.

⚙ Key Components of the Control Unit

The Control Unit relies on a few core components and internal signals to carry out its managerial functions:

Component	Description
Instruction Register (IR)	**Stores the instruction that was just fetched from memory. The CU decodes the *Opcode* part of this instruction to figure out what to do next.**
Clock/Timing Unit	**Provides the internal clock signal (a regular sequence of pulses) that synchronizes all operations. Every basic step (micro-operation) is timed by one or more clock pulses.**
Instruction Decoder	**A logic circuit that takes the instruction's *Opcode* from the IR and converts it into a complex, unique set of signals that specify which control signals should be activated.**

Control Signal Generator	**This matrix of logic circuits generates the final control signals (like READ, WRITE, ALU Select, Register Load) based on the decoded instruction, the current step in the execution cycle, and any input *Flags* (status indicators).**

Types of Control Units

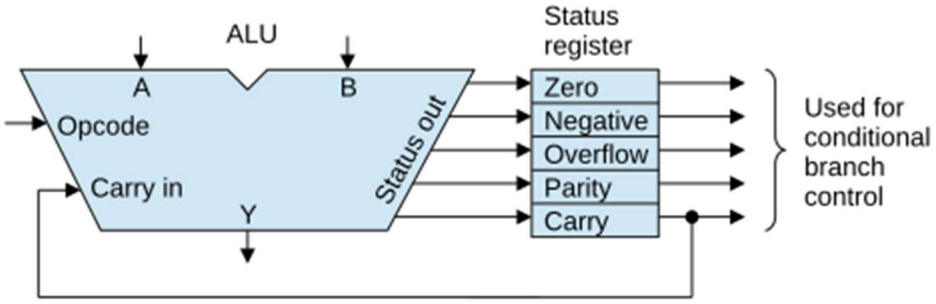
The internal implementation of the CU is classified into two main types:

Hardwired CU: The control signals are generated by fixed hardware logic circuits. It is faster but much harder to design or modify.

Microprogrammed CU: The control signals are stored as micro-instructions in a special, fast memory called Control Store. It is slower but flexible and easier to change (e.g., to add new instructions).

That is correct. The Von Neumann Model is fundamentally defined by these two properties, which together characterize it as a stored-program computer.

Arithmetic Logic Unit (ALU): Performs all arithmetic (addition, subtraction) and logical (AND, OR, comparison) operations.



The ALU is a combinational digital circuit responsible for performing all the arithmetic and logical operations requested by the program. It is the part of the CPU where the actual data processing takes place.

Function: It takes data from CPU registers, performs an operation based on a signal (the *opcode*) from the Control Unit, and stores the result back into a register or memory.

Structure: It consists of two primary units:

- **Arithmetic Unit (AU): Handles mathematical calculations.**
- **Logic Unit (LU): Handles comparative and Boolean operations.**

Operations: An ALU can perform a variety of functions, typically categorized into:

- **Arithmetic Operations: Addition, subtraction, multiplication, and division.**
- **Logic Operations: Boolean comparisons like AND, OR, NOT, XOR, and NAND.**
- **Bit-Shifting/Data Transfer Operations: Shifting the locations of bits to the right or left for purposes like multiplication or division.**

➕ Example of ALU Operation

Consider a simple instruction in a program: `ADD R1, R2` (Add the value in Register 2 to the value in Register 1, and store the result in Register 1).

Input: The Control Unit (CU) sends the data from Registers R1 (as Input A) and R2 (as Input B) to the ALU.

Control Signal (Opcode): The CU sends the control signal (`000` for "Add," for example) to the ALU's *Operation Selection Code* input, telling it to perform an addition.

Execution: The ALU's Arithmetic Unit performs the binary addition of Input A and Input B.

Output: The result is placed on the output data bus, along with Status Signals (like a "Carry-out" or "Zero" flag).

Storage: The CU directs the result from the ALU's output to be stored back into the destination register (R1 in this case).

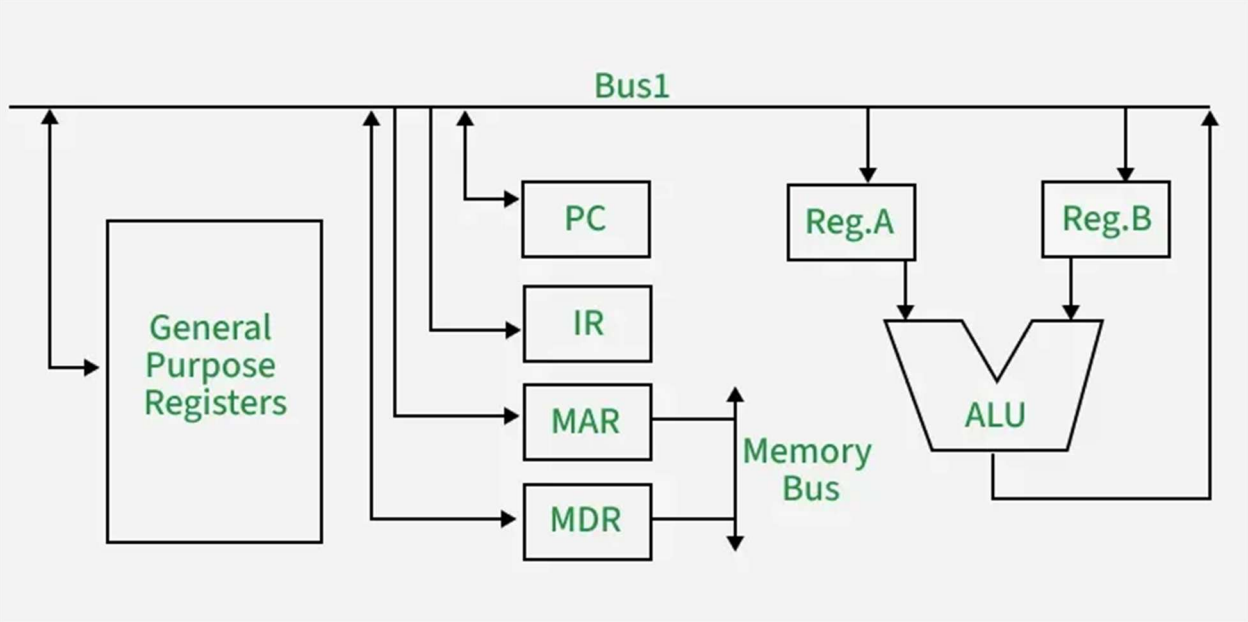
Block Diagram of an ALU

The diagram below illustrates the basic inputs and outputs of a generic Arithmetic Logic Unit.

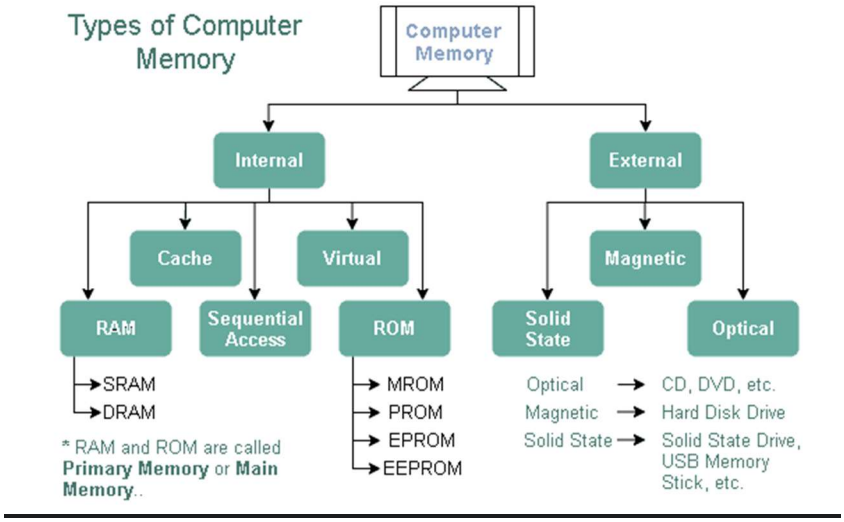
Signal Type	Description
Input Operands (A & B)	Two data values (binary numbers) the ALU will operate on.
Opcode	A control input from the Control Unit that selects which operation (Add, AND, Shift, etc.) the ALU should perform.
Status Inputs (e.g., Carry-in)	Extra information, such as the *Carry-out* bit from a previous operation, needed for multi-word arithmetic.
Result (Y)	The output of the operation (e.g., the sum of A and B).
Status Outputs	Signals (flags) that provide supplemental information about the result, such as Zero (result is 0), Negative, Carry-out, or Overflow.

****

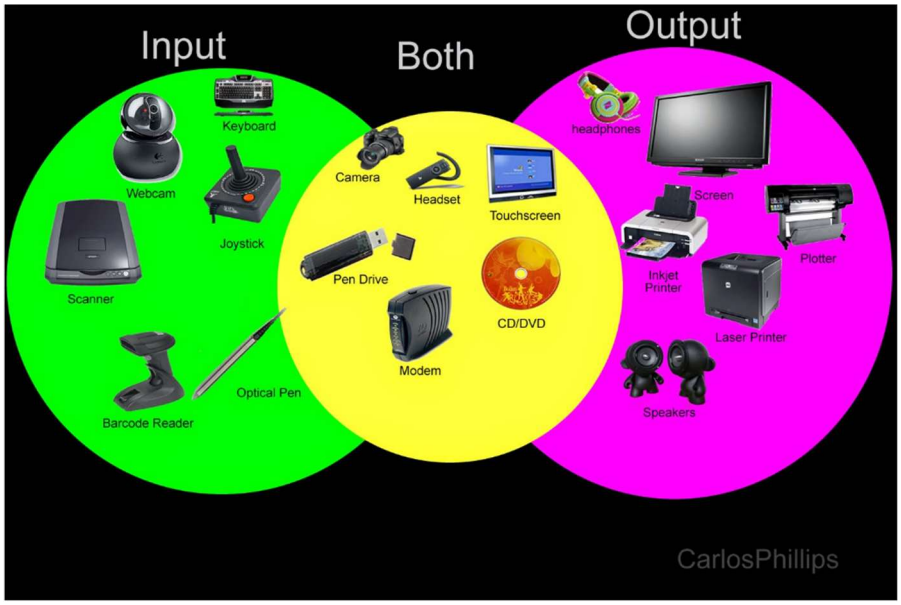
Registers: High-speed storage within the CPU (e.g., Program Counter, Memory Data Register) used to temporarily hold instructions and data during the Fetch-Decode-Execute (FDE) cycle.



Main Memory (RAM): A single, unified memory block that stores both instructions (code) and data (variables and input/output).



<u>Input/Output (I/O) System</u>: Interfaces with peripheral devices (keyboard, monitor, disk drives).



4.System Bus:

The bus is the communication pathway connecting the CPU, Memory, and I/O devices. Since all components share this single pathway, only one operation (either fetching an instruction or accessing data) can occur at any given moment. The bus is composed of :three parts:

1. Address Bus

Purpose: Specifies the source or destination of data on the data bus. When the CPU wants to read from or write to a memory location or an I/O port, it places the unique address of that location on the Address Bus.

Direction: Unidirectional (data travels only from the CPU/Control Unit to the Memory/I/O device).

Width: The width (number of lines) determines the maximum amount of physical memory the system can address. For example, a 32-bit address bus can address unique memory locations.

2. Data Bus

Purpose: Carries the actual data or program instructions being transferred between the CPU, memory, and I/O devices.

Direction: Bidirectional (data can flow both into and out of the CPU).

Width: The width (e.g., 32-bit, 64-bit) is a key factor in system performance, as it dictates how many bits can be transferred simultaneously in one operation.

3. Control Bus

Purpose: Carries control and timing signals used by the Control Unit to manage the activities of the entire system. It synchronizes the components and specifies the type of operation being performed.

Direction: Bidirectional (carries commands *from* the CPU and status signals *to* the CPU).

Examples of Signals:

> **Read/Write: Tells the memory unit whether the operation is a read (fetch) or a write (store).**
>
> **Bus Request: A component requests control of the bus.**
>
> **Clock: Synchronization signal.**

🌐 Bus Operation Example (Reading from Memory)

When the CPU needs to fetch a piece of data from memory, the three buses coordinate as follows:

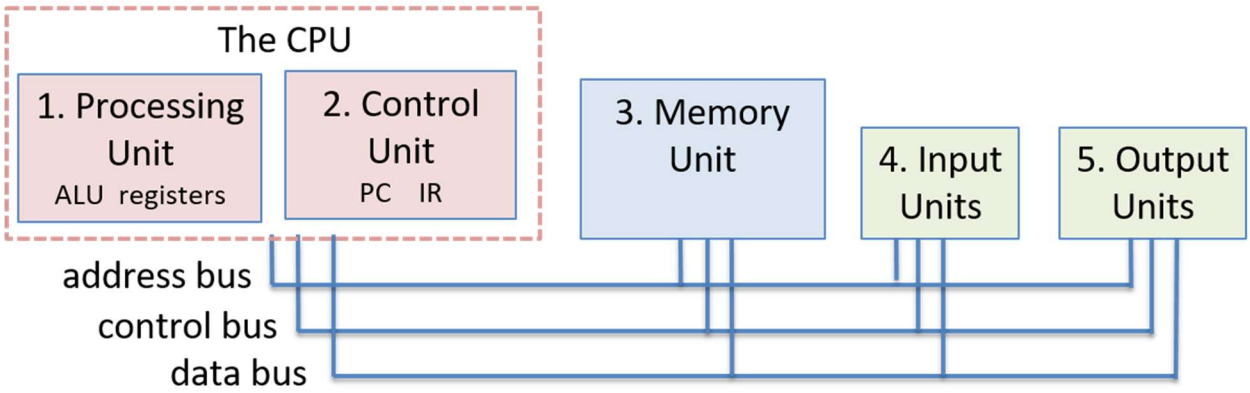
CPU Action: The CPU places the memory address of the data (e.g., `1234`) onto the Address Bus.

CPU Action: The CPU places a "READ" signal onto the Control Bus.

Memory Action: The Memory Unit receives the address and the READ signal, locates the data, and places the data (e.g., `00110011`) onto the Data Bus.

CPU Action: The CPU receives the data from the Data Bus and processes it.

between the CPU and Main Memory.



The Main Memory, or Memory Unit, is the core component of a computer's architecture responsible for storing both program instructions and data that the Central Processing Unit (CPU) is actively using.[1]

Main memory, which is typically Random Access Memory (RAM) in modern computers, acts as the CPU's direct, high-speed workspace.[2]

Characteristic	Description
Function	Stores the program (instructions) being run and the data currently being processed (variables, inputs). This is the basis of the stored-program concept.
Structure	It is organized as a large, ordered collection of storage locations (cells), each with a unique binary identifier called an address. The CPU uses these addresses to perform read (fetch) or write (store) operations.
Volatile	Main memory (RAM) is volatile, meaning its contents are lost when the computer is powered off. This is why data must be saved to secondary storage (like an SSD or HDD).
Speed	It is much faster than secondary storage but much slower than the CPU's internal registers and cache. This speed difference is a fundamental limitation known as the Von Neumann Bottleneck.

Example of Memory Unit Use

The main memory is crucial for the Fetch-Decode-Execute (F-D-E) cycle, which is how a CPU runs a program.[3]

Scenario: The CPU needs to fetch the next instruction.

Address Identification: The Program Counter (PC) register in the CPU holds the address of the next instruction, let's say address 2004.

Request: The Control Unit (CU) places the address 2004 onto the Address Bus and sends a READ command via the Control Bus.[4]

Memory Action: The Memory Unit receives the address and the READ command, locates the data at address 2004, and retrieves its contents (the instruction's binary code).

Data Transfer: The memory places the retrieved instruction onto the Data Bus.[5]

Instruction Fetch: The CPU receives the instruction from the Data Bus and stores it in the Instruction Register (IR) for decoding and execution.[6]

Analogy:

The Main Memory is like the desk of a person (the CPU) working on a project.

The CPU is the person doing the work.[7]

The Main Memory (RAM) is the desk where the worker keeps the project's instructions (the program code) and necessary materials (the data) to work with *right now*.[8]

The Secondary Storage (SSD/HDD) is the filing cabinet where the worker keeps *all* projects, only bringing the necessary files onto the desk when they are actively being worked on.

The Memory Address Register (MAR) and the Memory Data Register (MDR) are essential registers within the Central Processing Unit (CPU) that serve as the interface between the CPU and the Main Memory (RAM). They temporarily hold the critical information—the location and the data—needed for any memory access operation.

🔑 Memory Address Register (MAR)

The MAR holds the memory address of the location that the CPU wants to read data from or write data to.

Role: It specifies the exact cell in the main memory (RAM) that is being targeted.

Data Flow: The CPU's Control Unit places the address (e.g., 2004) onto the Address Bus, which is then used by the memory unit to locate the specific storage location.

🎁 Memory Data Register (MDR)

The MDR holds the data that is being transferred to or from the Main Memory. It is sometimes also called the Memory Buffer Register (MBR).

Role (Read): When the CPU is reading (fetching) data, the MDR temporarily holds the instruction or data item that has been retrieved from the memory location.

Role (Write): When the CPU is writing (storing) data, the MDR holds the data item that the CPU wants to save into the designated memory location.

Data Flow: Data moves between the MDR and memory via the Data Bus.

🔄 How MAR and MDR Work Together (The Fetch Cycle)

Here is a step-by-step example of how the MAR and MDR are used when the CPU fetches an instruction from memory:

Preparation: The Program Counter (PC) holds the address of the next instruction (e.g., address `2004`).

Load MAR: The Control Unit copies the address from the PC into the MAR (`MAR ← 2004`).

Address Transmission: The value in the MAR (`2004`) is sent across the Address Bus to the Main Memory.

Memory Read: The Control Unit issues a "READ" signal on the Control Bus. The memory unit finds address `2004` and retrieves the contents (the instruction, e.g., `ADD R1, R2`).

Data Transfer: The retrieved instruction is placed onto the Data Bus and then stored in the MDR (`MDR ← ADD R1, R2`).

Load Instruction Register: The instruction in the MDR is then moved to the Instruction Register (IR) for decoding and execution, and the cycle repeats.

The Control Unit (CU) is the manager or "brain" of the Central Processing Unit (CPU). Its primary function is to direct and coordinate the operation of the entire computer system by generating precise timing and control signals.



Computer Architecture and Computer Organization:

The terms Computer Architecture and Computer Organization are closely related but refer to different aspects of a computer system. The distinction lies in

whether an attribute is visible to the programmer (Architecture) or if it relates to the underlying physical implementation (Organization).

Here is a comparison of the two concepts:

Feature	Computer Architecture	Computer Organization
Definition	Refers to the attributes of a system visible to a programmer. These attributes have a direct impact on the logical execution of a program[3].	Refers to the operational units and their interconnections that realize the architectural specifications.
Examples	The Instruction Set , data types (e.g., number of bits used to represent data) , I/O mechanisms, and memory addressing techniques.	Control signals , interfaces between peripherals, and the memory technology used.

Design Issue	**A question of functionality (e.g., Will the computer have a multiply instruction?).**	**A question of hardware implementation (e.g., Will the multiply instruction use a special unit or repeatedly use the add unit?).**

The Von Neumann Architecture :

is the fundamental design for most modern, general-purpose computers, including PCs, laptops, and servers. It is based on the Stored-Program Concept, which means the computer's program instructions and the data the program operates on are stored .together in a single, unified memory unit.

The architecture was proposed by mathematician John von Neumann in 1945

Key Components of the Von Neumann Architecture

The Von Neumann model organizes the computer into four main functional blocks, which communicate through a single shared system bus.a design is referred to as the von Neumann architecture and is based on three key concepts:

1.Data and instructions are stored in a single read–write memory.

2.The contents of this memory are addressable by location, without regard to the type of data contained there.

3.Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to the next

Hardware and Software Approaches

The reasoning behind these concepts ca be summarizing here: There is a small set of basic logic components that can be combined in various ways to store binary data and to perform arithmetic and logical operations on that data. If there is a particular computation to be performed, a configuration of logic components designed specifically for that computation could be constructed. We can think of the process of connecting the various components in the desired configuration as a form of programming. The resulting "program" is in the form of hardware and is termed a hardwired program. Now consider this alternative. Suppose we construct a general-purpose configuration of arithmetic and logic functions. This set of hardware will perform various functions on data depending on control signals

applied to the hardware. In the original case of customized hardware, the system accepts data and produces results (Figure 3.1a). With general-purpose hardware, the system accepts data and control signals and produces results. Thus, instead of rewiring the hardware for each new program,the programmer merely needs to supply a new set of control signals. How shall control signals be supplied? The answer is simple but subtle. The entire program is actually a sequence of steps. At each step, some arithmetic or logical operation is performed on some data.
For each step, a new set of control signals is needed. Let us provide a unique code for each possible set of control signals, and let us add to the general-purpose hardware a segment that can accept a code and generate control signals (Figure 3.1b). Programming is now much easier. Instead of rewiring the hardware for each new program, all we need to do is provide a new sequence of codes. Each code is, in effect, an instruction, and part of the hardware interprets each instruction and generates control signals. To distinguish this new method of programming, a sequence of codes or instructions is called software.

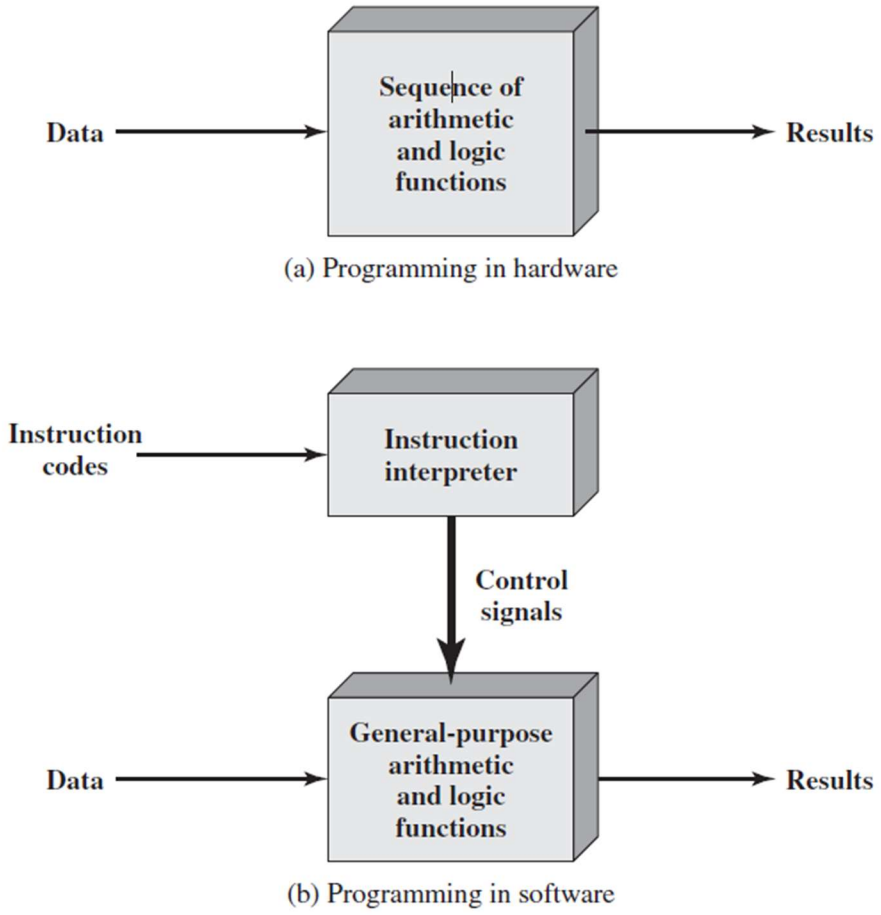
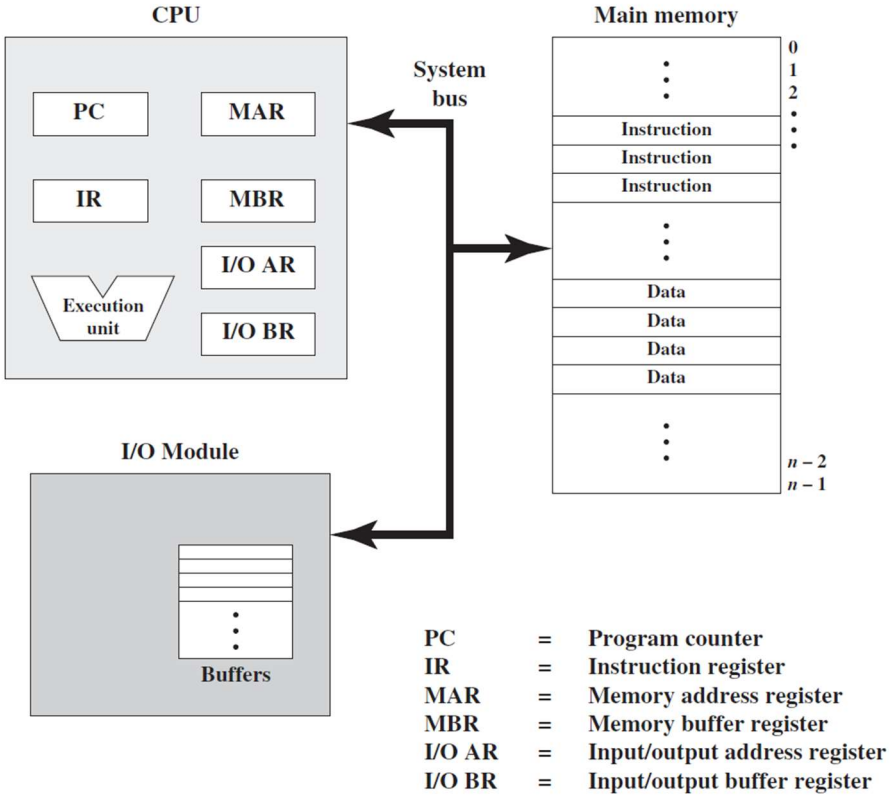


Fig 3.1 deference between hardware programming in Vs software programming

Figure below illustrates the top-level components and suggests the interactions among them to apply software. The CPU exchanges data with memory. For this purpose, it typically makes use of two internal

(to the CPU) registers: a memory address register (MAR), which specifies the address in memory for the next read or write, and a memory buffer register (MBR), which contains the data to be written into memory or receives the data read from memory. Similarly, an I/O address register (I/OAR) specifies a particular I/O device.An I/O buffer (I/OBR) register is used for the exchange of data between an I/O module and the CPU.

A memory module consists of a set of locations, defined by sequentially numbered addresses. Each location contains a binary number that can be interpreted as either an instruction or data. An I/O module transfers data from external devices to CPU and memory, and vice versa. It contains internal buffers for temporarily holding these data until they can be sent on.



COMPUTER FUNCTION

The basic function performed by a computer is execution of a program, which consists of a set of instructions stored in memory.
The processor does the actual work by executing instructions specified in the program.
This section provides an overview of the key elements of program execution.
In its simplest form,instruction processing consists of two steps:The processor reads (fetches) instructions from memory one at a time and executes each instruction.Program execution consists of repeating the process of instruction fetch and instruction execution.The instruction execution may

involve several operations and depends on the nature of the instruction , The processing required for a single instruction is called an instruction cycle. Using the simplified two-step description given previously, the instruction cycle is depicted in Figure 3.3.The two steps are referred to as the fetch cycle and the execute cycle.Program execution halts only if the machine is turned off,some sort of unrecoverable error occurs, or a program instruction that halts the computer is encountered .

Instruction Fetch and Execute:
At the beginning of each instruction cycle,the processor fetches an instruction from memory.In a typical processor,a register called the program counter (PC) holds the address of the instruction to be fetched next.Unless told otherwise,the processor always increments the PC after each instruction fetch so that it will fetch the next in struction in sequence (i.e., the instruction located at the next higher memory ad dress).

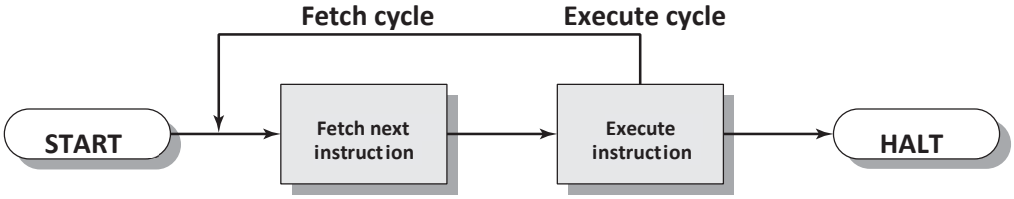


Figure 3.3 Basic Instruction Cycle

So, for example, consider a computer in which each instruction occupies one 16-bit word of memory.Assume that the program counter is set to location 300.The processor will next fetch the instruction at location 300. On succeeding instruction cycles, it will fetch instructions from locations 301, 302, 303, and so on.

Process Flow

The cycle repeats as follows:

1. **Initial State: The Program Counter (PC) holds the address of the next instruction to fetch (300).**
2. **Fetch Address: The address in the PC is transferred to the Memory Address Register (MAR).**
3. **Fetch Instruction: The MAR sends the address (300) to Memory, and the 16-bit instruction at that location is retrieved and placed into the MBR/IR.**
4. **Update PC: The PC is incremented by 1 (since each instruction is 1 word, which corresponds to 1 address unit). The PC now holds 301.**
5. **Execution: (Implied, not fully diagrammed) The instruction in the IR is decoded and executed.**
6. **Next Cycle: The process repeats, fetching the instruction from the new PC address (301), then 302, 303, and so on.**

This sequence may be altered, as explained presently.

The fetched instruction is loaded into a register in the processor known as the instruction register (IR). The instruction contains bits that specify the action the processor is to take. The processor interprets the instruction and performs the required action.

In general, these actions fall into four categories:

Processor Workflow: The Four Categories of Action

The workflow of a processor involves repeatedly fetching an instruction and then executing it, where execution involves one or more of the following four fundamental actions:

1. Processor-Memory Transfer

This action involves the movement of data between the main memory and the processor's registers.

Action: Transfer data from processor to memory (Write).

Action: Transfer data from memory to processor (Read).

Example: Fetching an instruction from memory into the Instruction Register, or loading an operand from a memory location into a general-purpose register.

2. Processor-I/O Transfer

This action manages the communication between the processor and peripheral devices (like keyboards, printers, or disk drives) via an I/O module.

Action: Transfer data from a peripheral device to the processor (Input).

Action: Transfer data from the processor to a peripheral device (Output).

3. Data Processing

This action is where the processor performs computations and manipulations on data.

Action: Perform some arithmetic operation on data (e.g., addition, subtraction, multiplication).

Action: Perform some logic operation on data (e.g., AND, OR, NOT, comparison).

4. Control (Program Flow Alteration)

This action alters the normal sequential execution flow of the program, which is typically controlled by the Program Counter (PC).

Action: An instruction specifies that the sequence of execution be altered.

Mechanism: The processor updates the Program Counter (PC) with a new address specified by the instruction.

Example: If the processor fetches an instruction from location 149 (a branch or jump instruction) that specifies the next instruction is at 182, the PC is set to 182. Consequently, on the next fetch cycle, the instruction is fetched from location 182 rather than the sequential next address (150).

The concept of computer software is inextricably linked to the Von Neumann Model because the architecture was the first to implement the stored-program concept, making software possible.

Software is generally classified into two main types:

---

1. System Software 💻

System software is designed to manage and control the computer hardware, providing a platform for application software to run.

Type	Description	Relation to Von Neumann Model
Operating System (OS)	**The core software (e.g., Windows, macOS, Linux) that manages memory, processes, files, and I/O devices.**	**The OS is the first program loaded into the Main Memory and is responsible for managing the entire computer as defined by the Von Neumann structure (CPU, Memory, I/O). It uses the Sequential Instruction Processing to schedule tasks and run other programs.**
Utility Programs	**Tools designed to help manage, maintain, and control computer resources (e.g., disk defragmenters, antivirus software, system cleaners).**	**These programs are treated as data, loaded into the shared memory space, and executed by the Control Unit just like any other program.**
Device Drivers	**Software that allows the OS to communicate with a specific piece of hardware (I/O device).**	**The drivers contain the instructions needed to control external I/O devices, coordinating data transfer between the Memory Unit and the device via the System Bus.**

Translator Software (Compilers, Assemblers, Interpreters) ⚙️

Translator software converts source code written by a programmer into the binary machine language (instructions) that the Central Processing Unit (CPU) understands. This conversion is necessary because the CPU can only process instructions stored in memory in binary form, as mandated by the Von Neumann architecture's stored-program concept.

Type	Description	Relation to Von Neumann Model
Compiler	**Translates an entire high-level program (like C++ or Java) into a single machine-code executable file before the program is run.**	**The compiler itself is loaded into the Main Memory and executed by the CPU. Its output is a binary file (machine code) which is then loaded into the unified memory space for the CPU to execute sequentially later.**
Assembler	**Translates programs written in Assembly Language (a symbolic, low-level language) directly into machine code.**	**It generates the exact binary instructions (the *Opcode* and *Operands*) that the CPU's Control Unit will fetch, decode, and execute using the System Bus.**
Interpreter	**Translates and executes one line of source code at a time during runtime (e.g., Python).**	**The interpreter program remains resident in Main Memory. For each line of code, it acts as an intermediary, executing the Fetch-Decode-Execute cycle on its *own* instructions to process and run the user's code.**

2. Application Software 🎮

Application software is designed for the user to perform specific tasks (e.g., web browsers, word processors, video games).

Type	Description	Relation to Von Neumann Model
Productivity Software	Programs used for office work (e.g., Microsoft Word, Excel).	The program's instructions and the user's document data are loaded into the unified memory (RAM). The CPU executes the application's instructions (e.g., "save a file") by coordinating the Control Unit and the ALU.
Entertainment Software	Video games, media players, etc.	High-performance applications heavily stress the architecture by frequently demanding both instructions (game logic) and data (textures, player scores) be fetched simultaneously from the Main Memory via the System Bus, which highlights the limitation known as the Von Neumann Bottleneck.

Each instruction must contain the information required by the processor for execu tion. Figure 10.1, shows the steps involved in instruction execution and, by implication, defines the elements of a machine instruction.These elements are as follows:

• Operation code: Specifies the operation to be performed (e.g., ADD, I/O). The operation is specified by a binary code, known as the operation code, or opcode.

• Source operand reference: The operation may involve one or more source operands,that is, operands that are inputs for the operation.

• Result operand reference:The operation may produce a result.

• Next instruction reference: This tells the processor where to fetch the next in struction after the execution of this instruction is complete. The address of the next instruction to be fetched could be either a real address or a virtual address, depending on the architecture.

GenerallyIn most cases, the next instruction to be fetched immediately follows the current instruction.In those cases,there is no explicit reference to the next instruction. When an explicit reference is needed, then the main memory or virtual memory address must be supplied.